

机

试

复

习

● 汇报人：何熙然 ●

目 录

01. 复习什么

02. 练习什么

03. 注意什么

PART. 01

课程内容回顾

复习什么

● 课程内容回顾 ●

以一个图书管理程序为例，回顾到目前为止我们学过的内容

```
#include <stdio.h>
#include <string.h>

// 定义结构体：存储书籍信息
typedef struct {
    char name[50]; // 书名
    float price;    // 价格
    int pages;      // 页数
} Book;

// 函数声明
void showMenu();           // 显示菜单
int getValidInt(int min, int max); // 获取有效整数（范围限制）
float calculateTotal(Book books[], int count); // 计算总价格
void sortBooksByPrice(Book books[], int count); // 按价格排序
```

```
#define MAX_SIZE 100 // 定义数组最大长度
#define PI 3.14159   // 定义圆周率常量
```

文件头部

- 头文件包含：用于引入标准库或自定义的头文件（#include）
- 宏定义：用于定义常量、简化代码（#define）
- 类型定义：用于给已有的数据类型起别名（typedef）
- 全局变量：定义在所有函数外部的变量（作用域为整个文件或项目）
- 函数声明：如果函数定义在main函数之后，需要在开头提前声明函数原型

课程内容回顾

常用标准库及库函数整理

头文件stdio.h

函数名	函数原型	函数功能
scanf	int scanf(const char *format,p);	按format格式输入数据给p所指向的内存单元, 文件结束返回EOF
printf	int printf(const char *format,args);	按format格式输出args的值
getchar	int getchar();	读取并返回字符直到遇上回车符
putchar	int putchar(char ch);	输出字符ch
gets	char *gets(char *str);	读入字符串到str指向的字符数组中, 直到读到回车符变成'\0'作为字符串的结束
puts	int puts(const char *str);	输出str指向的字符串, 将'\0'变成回车换行

头文件math.h

函数名	函数原型	函数功能
sin	double sin(double x);	返回sin(x)的值
cos	double cos(double x);	返回cos(x)的值
tan	double tan(double x);	返回tan(x)的值
abs	int abs(int x);	返回整数x的绝对值
fabs	double fabs(double x);	返回实数x的绝对值
floor	double floor(double x);	对x向下取整
ceil	double ceil(double x);	对x向上取整
pow	double pow(double x,double y);	返回x^y的值
log10	double log10(double x);	返回以10为底数, x为真数的对数
sqrt	double sqrt(double x);	返回x的正平方根

头文件string.h

函数名	函数原型	函数功能
strcpy	char *strcpy(char *x1,const char *x2)	将字符串x2复制到字符串x1中
strcat	char *strcat(char *x1,const char *x2)	将字符串x2连接到字符串x1后面
strcmp	char *strcmp(const char *x1,const char *x2)	按照字典顺序挨个字符比较两个字符串 (字母大小写敏感) x1<x2, 返回负数 x1=x2, 返回0 x1>x2, 返回正数
strupr	char *strupr(char *x)	将字符串x中的小写字母变成大写字母
strlwr	char *strlwr(char *x)	将字符串x中的大写字母变成小写字母
strlen	unsigned int char *strlen(const char *x)	返回字符串x的字符个数

头文件string.h

函数名	函数原型	函数功能
memset	void *memset(void *p,char ch,unsigned n);	将p为首地址的一片连续的n个字节内存单元都赋值为ch

```
char array[6];
memset(array, 'a', 6);

int num[6];
memset(num, 0, 6*sizeof(int));
```

● 课程内容回顾 ●

以一个图书管理程序为例，回顾到目前为止我们学过的内容

```
#include <stdio.h>
#include <string.h>

// 定义结构体：存储书籍信息
typedef struct {
    char name[50]; // 书名
    float price;    // 价格
    int pages;      // 页数
} Book;

// 函数声明
void showMenu();           // 显示菜单
int getValidInt(int min, int max); // 获取有效整数（范围限制）
float calculateTotal(Book books[], int count); // 计算总价格
void sortBooksByPrice(Book books[], int count); // 按价格排序
```

函数声明要注意：

- 函数声明必须放在第一次被调用的代码之前（通常在文件开头、main函数之前），否则编译器会报错“未声明的标识符”。
- 函数声明的返回类型、函数名、参数列表（类型 + 顺序）必须与函数定义完全一致，否则会导致编译错误或逻辑异常。
- 若函数参数为数组，声明时可省略数组长度（只需写类型 + []），因为数组传参本质是传地址。

```
// 声明时可省略参数名，只写类型（定义时必须有参数名）
int add(int, int); // 合法声明

// 定义（与声明的返回类型、参数类型/顺序一致）

int add(int a, int b) { // 参数名a、b在此处必须存在
    return a + b;
}
```

● 课程内容回顾 ●

以一个图书管理程序为例，回顾到目前为止我们学过的内容

```
int main() {  
    // 基础变量与初始化  
    int bookCount;  
    float avgPrice;  
  
    // 显示欢迎信息与菜单  
    printf("==== 简易图书管理系统 =====\n");  
    showMenu();  
  
    // 输入图书数量（带范围验证）  
    printf("\n请输入图书数量(1-5本):");  
    bookCount = getValidInt(1, 5);  
  
    // 结构体数组：存储多本图书信息  
    Book books[5];  
}
```

```
// 循环录入图书信息（for循环）  
for (int i = 0; i < bookCount; i++) {  
    printf("\n----- 录入第%d本图书 -----\n", i + 1);  
  
    // 字符串输入  
    printf("请输入书名: ");  
    scanf("%s", books[i].name);  
  
    // 带验证的数值输入（do-while循环）  
    do {  
        printf("请输入价格(0-100元):");  
        scanf("%f", &books[i].price);  
        if (books[i].price < 0 || books[i].price > 100) {  
            printf("价格超出范围，请重新输入! \n");  
        }  
    } while (books[i].price < 0 || books[i].price > 100);  
  
    do {  
        printf("请输入页数(10-1000页):");  
        scanf("%d", &books[i].pages);  
        if (books[i].pages < 10 || books[i].pages > 1000) {  
            printf("页数超出范围，请重新输入! \n");  
        }  
    } while (books[i].pages < 10 || books[i].pages > 1000);  
}
```

● 课程内容回顾 ●

以一个图书管理程序为例，回顾到目前为止我们学过的内容

```
#define N 19
int a[N] = {0};
```

```
int N; scanf("%d", &N);
int a[N] = {0};
```



```
struct Student arr[3] = {
    {"Alice", 18, 90.5},
    {"Bob", 19, 85.0},
    {"Charlie", 20, 92.3}
};
```

```
struct Student temp;
temp = arr[0];
arr[0] = arr[1];
arr[1] = temp;
```

```
if (arr[0] > arr[1]) {
    temp = arr[0];
    arr[0] = arr[1];
    arr[1] = temp;
}
```



数组要注意：

- 习惯性地为数组初始化一个定值，除非你很确定你在做什么
- 数组的长度必须是编译期常量，如字面量5、10，或用#define定义的宏常量
- 结构体数组可以整体地交换数据，但不能整体地比较

输入输出函数要注意（没有指针的情况下）：

- scanf()：除非是(%s)输入一个字符串，否则都要记得加 &
- printf()：都不用加 &
- scanf() 读入的字符串以空格或换行分隔；gets() 读入一行字符串直到换行符

课程内容回顾

以一个图书管理程序为例，回顾到目前为止我们学过的内容

格式字符	功能说明
%d, %i	用来输入有符号的十进制整数
%u	用来输入无符号的十进制整数
%o	用来输入无符号的八进制整数
%x, %X	用来输入无符号的十六进制整数（大小写作用相同）
%c	用来输入单个字符
%s	用来输入字符串
%f	用来输入实数，以小数形式输入
%e, %E	用来输入实数，以指数形式输入
%g, %G	用来输入实数，与 %f 和 %e 作用相同

转义字符	意义	ASCLL码值(十进制)
\a	响铃(BEL)	007
\b	退格(BS)，将当前位置移到前一位	008
\f	换页(FF)，将当前位置移到下页开头	012
\n	换行(LF)，将当前位置移到下一行开头	010
\r	回车(CR)，将当前位置移到本行开头	013
\t	水平制表(HT)（跳到下一个TAB位置）	009
\v	垂直制表(VT)	011
\'	代表一个单引号	039
\"	代表一个双引号字符	034
\\	代表一个反斜线字符\"	092
\?	代表一个问号	063
\0	空字符(NUL)	000
\ddd	1到3位八进制数所代表的任意字符	三位八进制
\xhh	十六进制所代表的任意字符	十六进制

● 课程内容回顾 ●

以一个图书管理程序为例，回顾到目前为止我们学过的内容

```
// 计算总价格与平均价格（函数调用）
float total = calculateTotal(books, bookCount);
avgPrice = total / bookCount;
// 条件判断（if-else分支）
printf("\n==== 图书统计结果 =====\n");
printf("图书总数: %d本\n", bookCount);
printf("总价: %.2f元\n", total);
printf("均价: %.2f元\n", avgPrice);
if (avgPrice > 50) {
    printf("提示: 本次图书均价较高\n");
} else if (avgPrice > 20) {
    printf("提示: 本次图书均价适中\n");
} else {
    printf("提示: 本次图书均价较低\n");
}
// 排序并展示（数组排序）
sortBooksByPrice(books, bookCount);
printf("\n==== 按价格从低到高排序 =====\n");
for (int i = 0; i < bookCount; i++) {
    printf("%d. 《%s》 - 价格: %.2f元, 页数: %d页\n",
        i + 1, books[i].name, books[i].price, books[i].pages);
}
```

```
// switch分支示例
printf("\n请选择操作(1-2): 1.查看最贵图书 2.查看最厚图书: ");
int choice = getValidInt(1, 2);

Book maxItem = books[0]; // 假设第一本是初始最大值
switch (choice) {
    case 1:
        for (int i = 1; i < bookCount; i++) {
            if (books[i].price > maxItem.price) {
                maxItem = books[i];
            }
        }
        printf("最贵的图书是《%s》, 价格: %.2f元\n", maxItem.name, maxItem.price);
        break;
    case 2:
        for (int i = 1; i < bookCount; i++) {
            if (books[i].pages > maxItem.pages) {
                maxItem = books[i];
            }
        }
        printf("最厚的图书是《%s》, 页数: %d页\n", maxItem.name, maxItem.pages);
        break;
    default:
        printf("无效选择\n");
}

printf("\n程序结束, 谢谢使用! \n");
return 0;
}
```

课程内容回顾

以一个图书管理程序为例，回顾到目前为止我们学过的内容

语句：

- if (if-else if-else)
- for (for(;;))
- while (do-while)
- switch
- break
- continue
- goto 

唯一需要提醒的是 {} 不能

忘加，与其担心忘记加，不

如所有都加上

```
if (条件) {  
    // 条件为真执行  
} else {  
    // 条件为假 (<0) 时执行  
}
```

```
if (条件1) { ... }  
else if (条件2) { ... }  
else if (条件3) { ... }  
else { ... } // 所有条件都不满足时执行
```

```
switch (表达式) {  
    case 常量1:  
        // 表达式值等于常量1时执行  
        break; // 跳出switch (若无break, 会继续执行下一个case)  
    case 常量2:  
        // 表达式值等于常量2时执行  
        break;  
    ...  
    default:  
        // 所有case都不匹配时执行 (可选)  
        break;  
}
```

```
for (初始化表达式; 循环条件; 迭代表达式) {  
    // 循环体 (条件为真时执行)  
}
```

```
while (循环条件) {  
    // 循环体 (条件为真时执行)  
}
```

```
do {  
    // 循环体  
} while (循环条件); // 注意末尾的分号
```

```
for (int i=0; i<10; i++) {  
    if (i==3) break; // 当i=3时跳出循环  
    printf("%d ", i); // 输出 0 1 2  
}
```

```
for (int i=0; i<5; i++) {  
    if (i==2) continue; // 跳过i=2的循环体剩余部分  
    printf("%d ", i); // 输出 0 1 3 4  
}
```

● 课程内容回顾 ●

以一个图书管理程序为例，回顾到目前为止我们学过的内容

```
// 显示功能菜单
void showMenu() {
    printf("功能说明: \n");
    printf("1. 录入图书信息 (书名、价格、页数) \n");
    printf("2. 自动计算总价与均价\n");
    printf("3. 按价格排序并展示\n");
    printf("4. 查询最贵/最厚图书\n");
}

// 获取指定范围内的有效整数
int getValidInt(int min, int max) {
    int num;
    do {
        scanf("%d", &num);
        if (num < min || num > max) {
            printf("输入错误, 请输入%d-%d之间的数: ", min, max);
        }
    } while (num < min || num > max);
    return num;
}
```

```
// 计算图书总价
float calculateTotal(Book books[], int count) {
    float sum = 0;
    for (int i = 0; i < count; i++) {
        sum += books[i].price;
    }
    return sum;
}

// 按价格升序排序 (冒泡排序)
void sortBooksByPrice(Book books[], int count) {
    for (int i = 0; i < count - 1; i++) {
        for (int j = 0; j < count - 1 - i; j++) {
            // 比较价格并交换
            if (books[j].price > books[j + 1].price) {
                Book temp = books[j];
                books[j] = books[j + 1];
                books[j + 1] = temp;
            }
        }
    }
}
```

排序算法

对一组数据按照特定顺序进行排列。排序算法中的数据类型可以是整数、浮点数、字符或字符串等。排序的判断规则可根据需求设定，如数字大小、字符 ASCII 码顺序或自定义规则。

按照数字大小排序整数



按照字典序排序字符串



按照自定义规则排序字符串

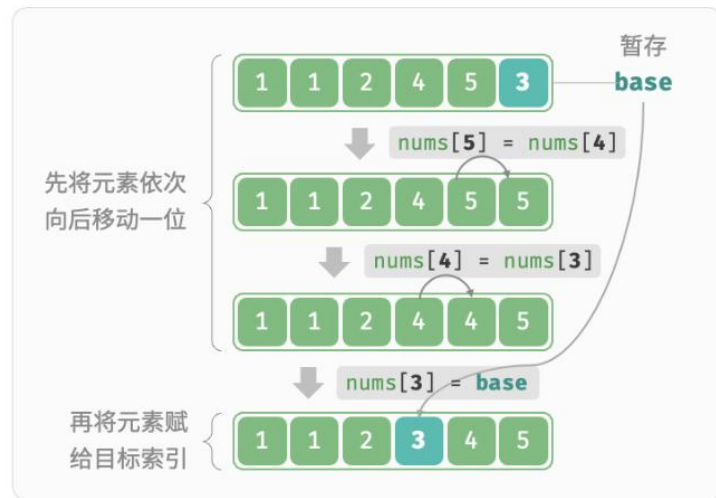
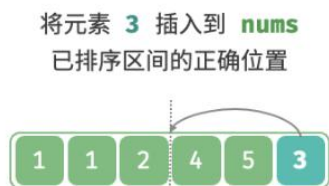


● 插入排序

● 选择排序

● 冒泡排序

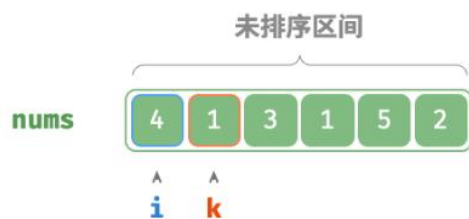
插入排序



已排序区间



选择排序



第 1 轮选择:

遍历未排序区间, 选择最小元素 `nums[k]`



第 1 轮选择:

遍历未排序区间, 选择最小元素 `nums[k]`

交换最小元素 `nums[k]` 和未排序区间的首个元素 `nums[i]`

不稳定



执行第一轮选择



完成第一轮选择后,
两个元素 4 的相对位置发生了改变

● 课程内容回顾 ●

样例代码

```
/* 插入排序 */
void insertionSort(int nums[], int size) {
    // 外循环: 已排序区间为 [0, i-1]
    for (int i = 1; i < size; i++) {
        int base = nums[i], j = i - 1;
        // 内循环: 将 base 插入到已排序区间 [0, i-1] 中的正确位置
        while (j >= 0 && nums[j] > base) {
            // 将 nums[j] 向右移动一位
            nums[j + 1] = nums[j];
            j--;
        }
        // 将 base 赋值到正确位置
        nums[j + 1] = base;
    }
}
```

```
/* 选择排序 */
void selectionSort(int nums[], int n) {
    // 外循环: 未排序区间为 [i, n-1]
    for (int i = 0; i < n - 1; i++) {
        // 内循环: 找到未排序区间内的最小元素
        int k = i;
        for (int j = i + 1; j < n; j++) {
            if (nums[j] < nums[k])
                k = j; // 记录最小元素的索引
        }
        // 将该最小元素与未排序区间的首个元素交换
        int temp = nums[i];
        nums[i] = nums[k];
        nums[k] = temp;
    }
}
```

课程内容回顾

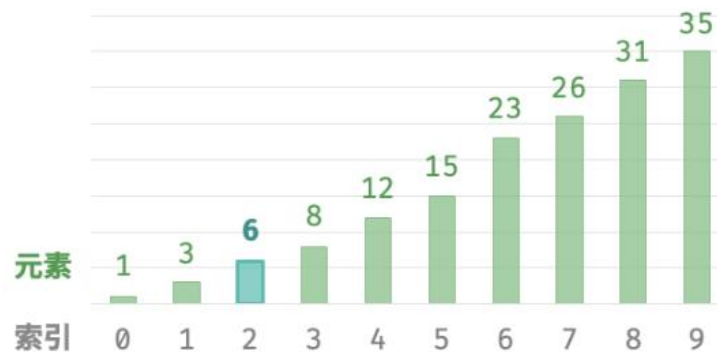
冒泡排序

```
/* 冒泡排序（标志优化）*/
void bubbleSortWithFlag(int nums[], int size) {
    // 外循环：未排序区间为 [0, i]
    for (int i = size - 1; i > 0; i--) {
        int flag = 0;
        // 内循环：将未排序区间 [0, i] 中的最大元素交换至该区间的右端
        for (int j = 0; j < i; j++) {
            if (nums[j] > nums[j + 1]) {
                int temp = nums[j];
                nums[j] = nums[j + 1];
                nums[j + 1] = temp;
                flag = 1;
            }
        }
        if (!flag)
            break;
    }
}
```



二分查找

二分查找 (binary search) 是一种基于分治策略的高效搜索算法。它利用数据的有序性，每轮缩小一半搜索范围，直至找到目标元素或搜索区间为空为止。



使用柱状图表示数组

nums



输入有序数组 **nums**，查找目标元素 **target = 6**

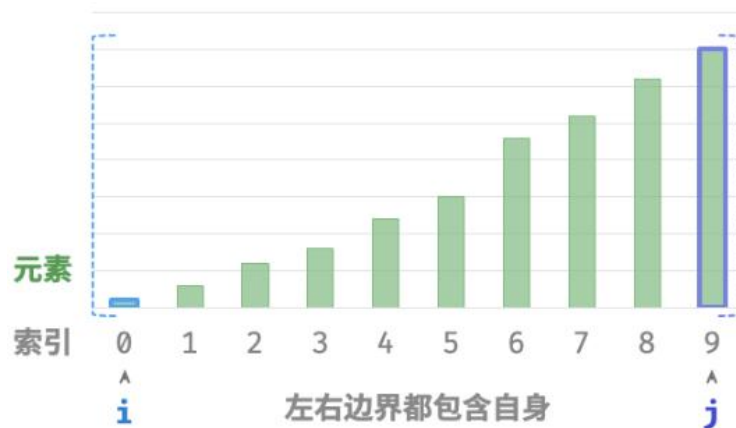
二分查找

```
/* 二分查找（双闭区间） */
int binarySearch(int nums[], int len, int target) {
    // 初始化双闭区间 [0, n-1]，即 i, j 分别指向数组首元素、尾元素
    int i = 0, j = len - 1;
    // 循环，当搜索区间为空时跳出（当 i > j 时为空）
    while (i <= j) {
        int m = i + (j - i) / 2; // 计算中点索引 m
        if (nums[m] < target) // 此情况说明 target 在区间 [m+1, j] 中
            i = m + 1;
        else if (nums[m] > target) // 此情况说明 target 在区间 [i, m-1] 中
            j = m - 1;
        else // 找到目标元素，返回其索引
            return m;
    }
    // 未找到目标元素，返回 -1
    return -1;
}
```

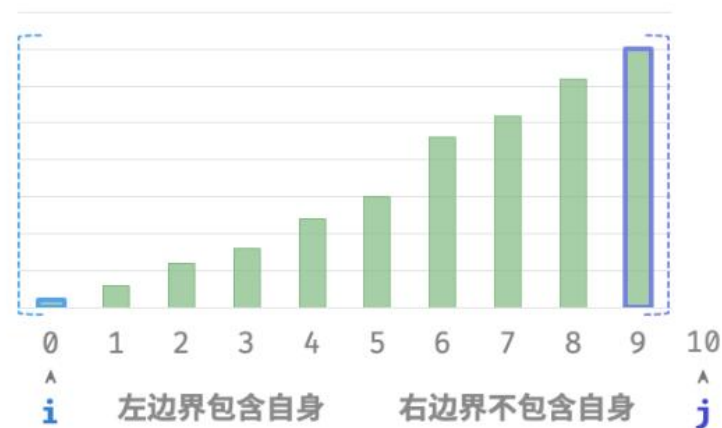
```
/* 二分查找（左闭右开区间） */
int binarySearchLCRO(int nums[], int len, int target) {
    // 初始化左闭右开区间 [0, n)，即 i, j 分别指向数组首元素、尾元素+1
    int i = 0, j = len;
    // 循环，当搜索区间为空时跳出（当 i = j 时为空）
    while (i < j) {
        int m = i + (j - i) / 2; // 计算中点索引 m
        if (nums[m] < target) // 此情况说明 target 在区间 [m+1, j) 中
            i = m + 1;
        else if (nums[m] > target) // 此情况说明 target 在区间 [i, m) 中
            j = m;
        else // 找到目标元素，返回其索引
            return m;
    }
    // 未找到目标元素，返回 -1
    return -1;
}
```

● 课程内容回顾 ●

二分查找



搜索区间：双闭区间 $[i, j]$
初始化指针： $i = 0, j = n - 1$
循环终止条件： $i > j$
缩小区间操作： $i = m + 1, j = m - 1$



搜索区间：左闭右开区间 $[i, j)$
初始化指针： $i = 0, j = n$
循环终止条件： $i \geq j$
缩小区间操作： $i = m + 1, j = m$

二分插入

给定一个长度为 n 的有序数组 `nums[]` 和一个元素 `target`，数组不存在重复元素。现将 `target` 插入数组 `nums[]` 中，并保持其有序性。若数组中已存在元素 `target`，则插入到其左方。请返回插入后 `target` 在数组中的索引。



输入有序数组 `nums`，目标元素 `target` = 6
则 `target` 的插入点索引为 2

二分插入

如果想复用上一节的二分查找代码，则需要回答以下两个问题：

1. 当数组中包含 `target` 时，插入点的索引是否是该元素的索引？
2. 当数组中不存在 `target` 时，插入点是哪个元素的索引？



输入有序数组 `nums`，目标元素 `target` = 6
则 `target` 的插入点索引为 2

二分插入

1. 当数组中包含 `target` 时，插入点的索引是否是该元素的索引？

答：题目要求将 `target` 插入到相等元素的左边，这意味着新插入的 `target` 替换了原来 `target` 的位置。也就是说，当数组包含 `target` 时，插入点的索引就是该 `target` 的索引。



输入有序数组 `nums`，目标元素 `target` = 6
则 `target` 的插入点索引为 2

二分插入

2. 当数组中不存在 `target` 时，插入点是哪个元素的索引？

答：当 `nums[m] < target` 时 `i` 移动，这意味着 `i` 在向大于等于 `target` 的元素靠近。同理，`j` 始终在向小于等于 `target` 的元素靠近。因此二分结束时一定有：`i` 指向首个大于 `target` 的元素，`j` 指向首个小于 `target` 的元素。易得当数组不包含 `target` 时，插入索引为 `i`。



输入有序数组 `nums`，目标元素 `target` = 6
则 `target` 的插入点索引为 2

二分插入

假如存在重复的元素，应该怎么办？

- 先二分，再线性
- 第一次找到 `target` 时，执行 $j = m - 1$ 。循环完成后，`i` 指向最左边的 `target`，`j` 指向首个小于 `target` 的元素，因此索引 `i` 就是插入点。



循环二分查找：

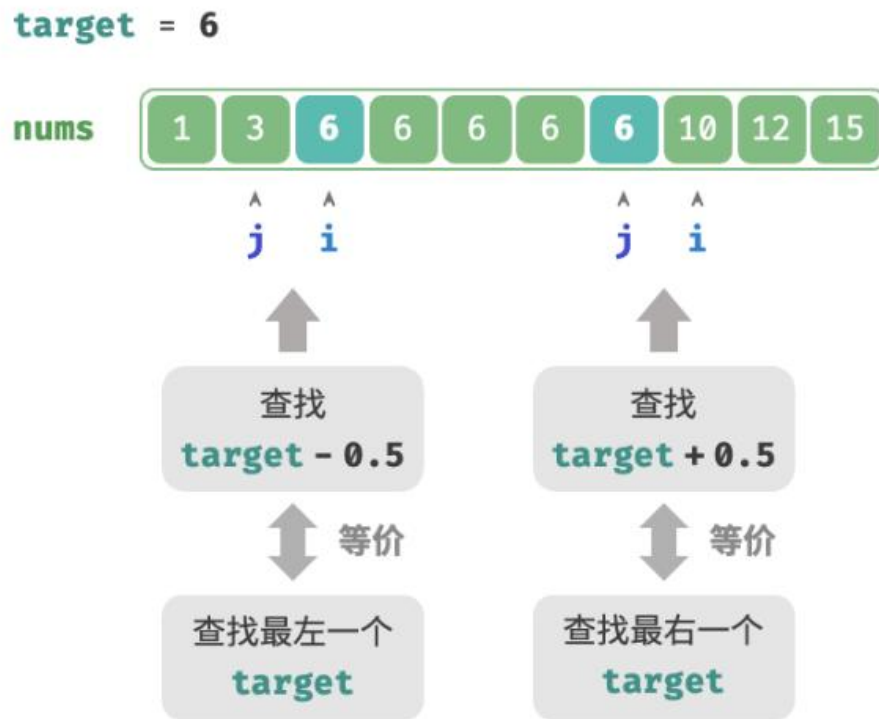
1. 计算中点 $m = (i + j) / 2$

2. $\because \text{nums}[m] == \text{target}$
 $\therefore$ 执行 $j = m - 1$ 缩小搜索区间

二分插入

当数组不包含 `target` 时，最终 `i` 和 `j` 会分别指向首个大于、小于 `target` 的元素。因此，我们可以构造一个数组中不存在的元素，用于查找左右边界。

- 查找最左一个 `target`：可以转化为查找 `target - 0.5`，并返回 `i`。
- 查找最右一个 `target`：可以转化为查找 `target + 0.5`，并返回 `j`。



PART. 02

常见题型

练习什么

两种可能出现的编程题（猜的）



模拟

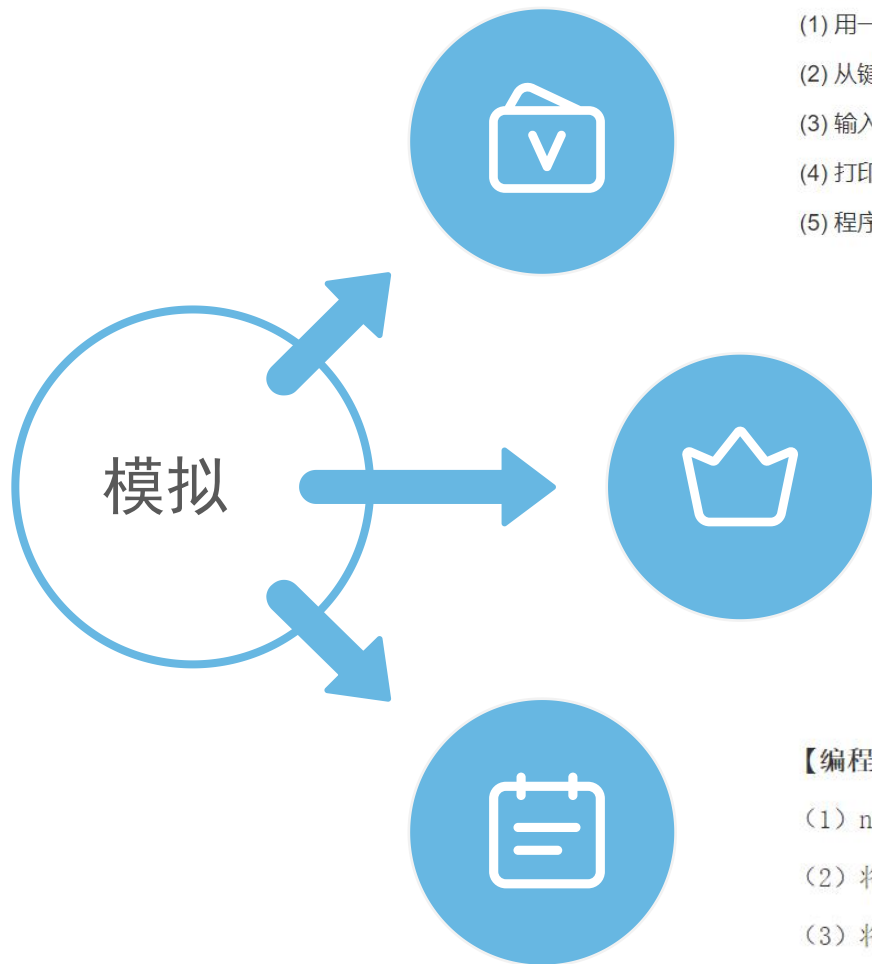
模拟算法是一种通过模仿现实场景或问题的运行过程来求解问题的方法。它不依赖复杂的数学推导或数据结构优化，而是按照问题的具体规则、步骤或流程，一步步“复现”过程，最终得到结果。



贪心

贪心算法是一种每次决策都选择当前局部最优解，最终期望得到全局最优解的启发式算法，核心是“目光短浅”但符合问题的贪心选择性质。

常见题型



【编程要求】

- (1) 用一个结构体类型表示24小时制的时间，包含时、分、秒等3个整型成员；
- (2) 从键盘输入时间（时、分、秒）到结构体变量中，如输入的时间不符合24小时制时间取值要求，在不退出程序的情况下可以重新输入时间；
- (3) 输入表示秒数的整数用于调整时间，负数表示从当前时间回退的秒数，正数表示从当前时间前进的秒数，0表示退出程序；
- (4) 打印调整前后的时间，时、分、秒用减号连接，分别以两位整数输出，不足两位的前面补0（可使用格式%02d）；
- (5) 程序运行时，能够多次进行时间的调整（皆以原输入时间为基准），输入调整的秒数为0时退出程序。

【问题描述】

- (1) 用牛顿迭代法求解方程在1.5附近的根，精度小于 10^{-6}
- (2) 用二分法求解方程在 $[-3, 3]$ 内的根，精度小于 10^{-6}
- (3) 统计两种方法的迭代次数（主要循环的执行次数）
- (4) 比较并分析两种求解方法的结果与特点

【编程要求】任一个大于2的偶数都可以写成两个素数之和。设计程序验证n以内的偶数满足哥德巴赫猜想。

- (1) n由键盘输入，并判断其是否为大于2的偶数
- (2) 将从4开始到不大于n的偶数全部写成两个素数之和的形式输出
- (3) 将每个偶数所有的可能拆解都写出来，但不要把两个素数交换位置的也打印出来。

常见题型

#159. 【清华集训2015】多边形下海

好评 差评 [-593]

描述 提交 自定义测试

附件下载 统计

在一个二维空间中，给定一个凸多边形，假设该凸多边形的面密度为 ρ ，放入一个水平面为 $y = 0$ 的无穷大的水池当中，水的密度为 1，求该凸多边形在静止并且势能到达最低点的情况下，该凸多边形逆时针旋转了多少？

势能包括凸多边形本身的重力势能和浮力对其做功产生的浮力势能。

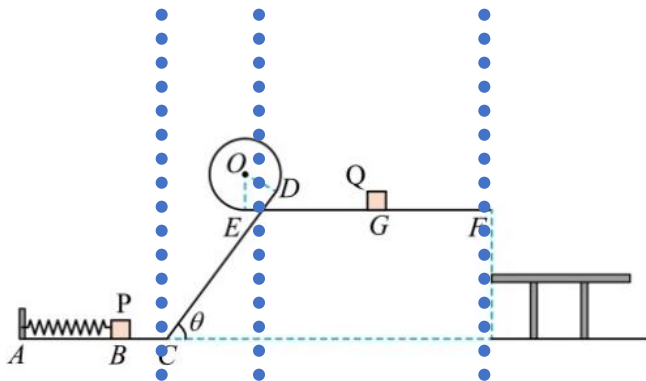
输入格式

第一行一个数 N ，表示数据组数。对于每组数据：第一行两个数，整数 n 和浮点数 ρ ，表示多边形点的个数和多边形的密度。接下来 n 行，每行两个浮点数 x, y 逆时针表示凸包上的所有点。

输出格式

对于每一组数据输出一行， θ ，表示凸多边形达到势能的最低点的时候，其逆时针旋转的弧度是多少， $0 \leq \theta < 2\pi$ 。

本题包含 Special Judge，对于你的输出允许 10^{-6} 的误差，请输出小数部分位数尽量多的数（但小数位数不要超过 15）。



495. 提莫攻击

已解答

简单 相关标签 相关企业 Aa

在《英雄联盟》的世界中，有一个叫“提莫”的英雄。他的攻击可以让敌方英雄艾希（编者注：寒冰射手）进入中毒状态。

当提莫攻击艾希，艾希的中毒状态正好持续 `duration` 秒。

正式地讲，提莫在 `t` 发起攻击意味着艾希在时间区间 `[t, t + duration - 1]`（含 `t` 和 `t + duration - 1`）处于中毒状态。如果提莫在中毒影响结束前再次攻击，中毒状态计时器将会重置，在新的攻击之后，中毒影响将会在 `duration` 秒后结束。

给你一个非递减的整数数组 `timeSeries`，其中 `timeSeries[i]` 表示提莫在 `timeSeries[i]` 秒时对艾希发起攻击，以及一个表示中毒持续时间的整数 `duration`。

返回艾希处于中毒状态的总秒数。

6. Z 字形变换

已解答

中等 相关标签 相关企业 Aa

将一个给定字符串 `s` 根据给定的行数 `numRows`，以从上往下、从左到右进行 Z 字形排列。

比如输入字符串为 "PAYPALISHIRING" 行数为 3 时，排列如下：

```
P A H N
A P L S I I G
Y I R
```

之后，你的输出需要从左往右逐行读取，产生出一个新的字符串，比如："PAHNAPLSIIGYIR"。

请你实现这个将字符串进行指定行数变换的函数：

```
string convert(string s, int numRows);
```

常见题型

01



阅读题目

- 阅读题中的每句话，不要遗漏信息
- 理解题目的要求，不要根据自己的经验来判断

02



翻译题目

- 用什么结构类型保存输入数据
- 输出时采用什么方式输出
- 将自然语言翻译成计算机的语句

03



实现题目

- 根据翻译的结果一步步完成程序
- 如果提交时发现错误，重复前面的步骤

常见题型

回文日期

题目描述

2020 年春节期间，有一个特殊的日期引起了大家的注意：2020 年 2 月 2 日。因为如果将这个日期按“yyyymmdd”的格式写成一个 8 位数是 20200202，恰好是一个回文数。我们称这样的日期是回文日期。

有人表示 20200202 是“千年一遇”的特殊日子。对此小明很不认同，因为不到 2 年之后就是下一个回文日期：20211202 即 2021 年 12 月 2 日。

也有人表示 20200202 并不仅仅是一个回文日期，还是一个 ABABBABA 型的回文日期。对此小明也不认同，因为大约 100 年后就能遇到下一个 ABABBABA 型的回文日期：21211212 即 2121 年 12 月 12 日。算不上“千年一遇”，顶多算“千年两遇”。

给定一个 8 位数的日期，请你计算该日期之后下一个回文日期和下一个 ABABBABA 型的回文日期各是哪一天。

输入描述

输入包含一个八位整数 N ，表示日期。

对于所有评测用例， $10000101 \leq N \leq 89991231$ ，保证 N 是一个合法日期的 8 位数表示。

输出描述

输出两行，每行 1 个八位数。第一行表示下一个回文日期，第二行表示下一个 ABABBABA 型的回文日期。

查看题目要求：

计算给定日期之后的下一个回文日期和下一个 ABABBABA 型的回文日期分别是什么？

翻译

- 回文数怎么判断？
- ABABBABA怎么判断？
- 如何判断一个日期是否合理？

具体的实现方式：

日期是按照年月日遍历；还是当成一个大整数遍历，之后再判断是否是一个合法的日期。

常见题型

回文日期

题目描述

2020 年春节期间，有一个特殊的日期引起了大家的注意：2020 年 2 月 2 日。因为如果将这个日期按“yyyymmdd”的格式写成一个 8 位数是 20200202，恰好是一个回文数。我们称这样的日期是回文日期。

有人表示 20200202 是“千年一遇”的特殊日子。对此小明很不认同，因为不到 2 年之后就是下一个回文日期：20211202 即 2021 年 12 月 2 日。

也有人表示 20200202 并不仅仅是一个回文日期，还是一个 ABABBABA 型的回文日期。对此小明也不认同，因为大约 100 年后就能遇到下一个 ABABBABA 型的回文日期：21211212 即 2121 年 12 月 12 日。算不上“千年一遇”，顶多算“千年两遇”。

给定一个 8 位数的日期，请你计算该日期之后下一个回文日期和下一个 ABABBABA 型的回文日期各是哪一天。

输入描述

输入包含一个八位整数 N ，表示日期。

对于所有评测用例， $10000101 \leq N \leq 89991231$ ，保证 N 是一个合法日期的 8 位数表示。

输出描述

输出两行，每行 1 个八位数。第一行表示下一个回文日期，第二行表示下一个 ABABBABA 型的回文日期。

```
int year, month, day; // 4 + 2 + 2, 将八位输入分解成年月日
for (; year <= 9999; year++) {
    // 需要判断是否是闰年
    for (; month <= 12; month++) {
        // 需要判断每个月的天数
        for (; day <= 31; day++) {
            // 用一个compare()函数判断是否是回文或者ABABBABA
        }
    }
}
```

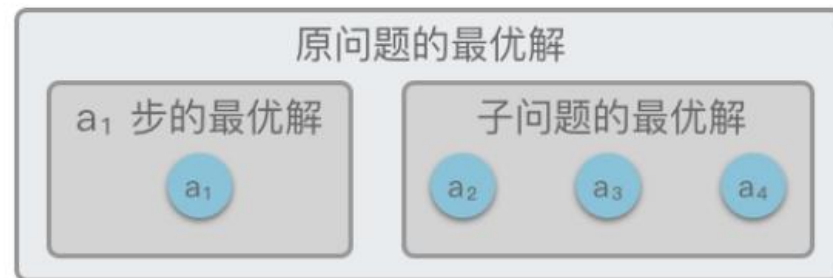
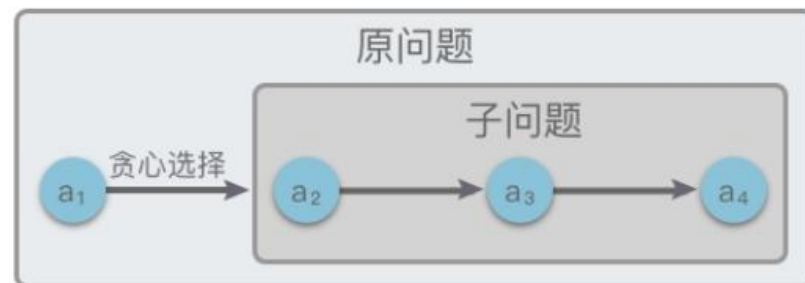
```
int number;
for (; number <= 100000000; number++) {
    // 判断是否是一个合法的日期
    // 判断是否是回文数
    // 判断是否是ABABBABA型数
}
```

贪心算法

贪心算法适用于一类特殊问题：只要每一步都做出当前最优选择，最终就能得到整体最优解或近似最优解。但并非所有问题都适用贪心算法。

通常，能用贪心算法解决的问题需同时满足两个条件：

- 贪心选择性质：全局最优解可以通过一系列局部最优（贪心）选择获得。
- 最优子结构性质：问题的最优解包含其子问题的最优解。



贪心算法

贪心算法的难点在于如何证明其选择策略能得到全局最优解。常见的两种证明方法：

- 数学归纳法（应该很熟悉）
- 交换论证法：假设存在更优解，通过交换局部选择，如果不会得到更优结果，则当前贪心解为最优。（有点类似证明不等式中的调整法）

实际做题时往往先用贪心思路做一遍，看看局部最优能否推出全局最优。尝试构造反例，如果找不到局部最优导致全局最优失败的例子，基本可以用贪心法。

§ 【问题描述】

有一个背包最大可以容纳 x 重量的物品，现在有 n 个物品，均有各自的重量和价值。问如何将这些物品装入这个背包里，使得背包里物品的价值最大，且其重量不能超过背包的最大容量。

问题说明：

1) 0-1背包问题：每种物品仅1个，不可拆分，只能全部装入或不装入；

§ 编程要求

1) 用贪心法分别按价值和单位重量价值（物品的价值除以物品的重量）求解0-1背包问题，不需要得到最优解；

2) 输入 n 个（ $3 \leq n \leq 10$ ）物品的编号、重量和价值、以及背包可以容纳的重量 x ，可以不按顺序输入，但程序里须根据需要进行排序。

输入提示语和输出格式严格和课本保持一致。

贪心算法

1. 找零问题（特定货币体系下）

场景：用最少的硬币凑出指定金额，硬币面额为任意组合（非现实中的“贪心友好型”面额）。

看似可行的贪心思路：每次优先选最大面额的硬币，直到凑够金额。

错误原因：特定面额组合下，局部选最大面额会导致总硬币数更多。

例子：

- 硬币面额：[1, 3, 4]，目标金额 6。
- 贪心策略：先选 4，剩余 2 用两个 1，总硬币数 3 (4+1+1)。
- 最优解：3+3，总硬币数 2，显然更优。

关键：只有当货币体系满足“每一种面额是下一种的倍数”（如人民币 100、50、10、1），贪心才有效；否则必须用动态规划。

3. 股票买卖：多次交易最大化利润（含冷冻期）

场景：给定股票价格数组，可多次买卖，但卖出后次日不能买入（冷冻期），求最大利润。

看似可行的贪心思路：只要当天价格高于前一天，就买入并次日卖出（低买高卖），避开冷冻期。

错误原因：连续上涨时，贪心会频繁买卖触发冷冻期，错失更大利润。

例子：

- 价格：[1, 2, 3, 4]（连续上涨）。
- 贪心策略：
 - 第 1 天买，第 2 天卖（利润 1），冷冻期第 3 天，第 4 天买但无法卖出，总利润 1。
- 最优解：
 - 第 1 天买，第 4 天卖（利润 3），无冷冻期影响，利润更高。

关键：贪心只看相邻两天的利润，忽略了长期持有可能获得的更大收益，必须用动态规划跟踪“持有”“卖出”“冷冻”三种状态。

贪心算法

455. 分发饼干

已解答 

简单

 相关标签

 相关企业

Ax

假设你是一位很棒的家长，想要给你的孩子们一些小饼干。但是，每个孩子最多只能给一块饼干。

对每个孩子 i ，都有一个胃口值 $g[i]$ ，这是能让孩子们满足胃口的饼干的最小尺寸；并且每块饼干 j ，都有一个尺寸 $s[j]$ 。如果 $s[j] \geq g[i]$ ，我们可以将这个饼干 j 分配给孩子 i ，这个孩子会得到满足。你的目标是满足尽可能多的孩子，并输出这个最大数值。

示例 1:

输入: $g = [1,2,3]$, $s = [1,1]$

输出: 1

解释:

你有三个孩子和两块小饼干，3 个孩子的胃口值分别是: 1,2,3。

虽然你有两块小饼干，由于他们的尺寸都是 1，你只能让胃口值是 1 的孩子满足。

所以你应该输出 1。

示例 2:

输入: $g = [1,2]$, $s = [1,2,3]$

输出: 2

解释:

你有两个孩子和三块小饼干，2 个孩子的胃口值分别是 1,2。

你拥有的饼干数量和尺寸都足以让所有孩子满足。

所以你应该输出 2。

提示:

- $1 \leq g.length \leq 3 \times 10^4$
- $0 \leq s.length \leq 3 \times 10^4$
- $1 \leq g[i], s[j] \leq 2^{31} - 1$

贪心算法

455. 分发饼干

已解答 

简单

 相关标签

 相关企业

Ax

假设你是一位很棒的家长，想要给你的孩子们一些小饼干。但是，每个孩子最多只能给一块饼干。

对每个孩子 i ，都有一个胃口值 $g[i]$ ，这是能让孩子们满足胃口的饼干的最小尺寸；并且每块饼干 j ，都有一个尺寸 $s[j]$ 。如果 $s[j] \geq g[i]$ ，我们可以将这个饼干 j 分配给孩子 i ，这个孩子会得到满足。你的目标是满足尽可能多的孩子，并输出这个最大数值。

解题思路：

为了让尽可能多的孩子得到满足，且每块饼干只能分配给一个孩子，我们应优先用小尺寸的饼干满足胃口较小的孩子，将大尺寸的饼干留给胃口较大的孩子。

基于贪心思想，具体做法如下：

先将孩子的胃口数组 g 和饼干尺寸数组 s 分别从小到大排序。然后，依次为每个孩子分配能够满足其胃口的最小尺寸饼干。

贪心算法

455. 分发饼干

已解答

简单

🔖 相关标签

🏢 相关企业

Ax

假设你是一位很棒的家长，想要给你的孩子们一些小饼干。但是，每个孩子最多只能给一块饼干。

对每个孩子 i ，都有一个胃口值 $g[i]$ ，这是能让孩子们满足胃口的饼干的最小尺寸；并且每块饼干 j ，都有一个尺寸 $s[j]$ 。如果 $s[j] \geq g[i]$ ，我们可以将这个饼干 j 分配给孩子 i ，这个孩子会得到满足。你的目标是满足尽可能多的孩子，并输出这个最大数值。

```
int findContentChildren(int g[], int gSize, int s[], int sSize) {
    // 冒泡排序孩子胃口和饼干尺寸
    bubbleSort(g, gSize);
    bubbleSort(s, sSize);

    int index_g = 0; // 当前待满足的孩子索引
    int index_s = 0; // 当前待尝试的饼干索引
    int res = 0;      // 满足的孩子数量

    while (index_g < gSize && index_s < sSize) {
        if (g[index_g] <= s[index_s]) {
            res++;
            index_g++;
            index_s++;
        } else {
            index_s++;
        }
    }

    return res;
}
```

PART. 03

机试注意点

Pay Attention

平台可能地评测方式：

1. 完全匹配

系统默认的匹配策略，这是最为常用的匹配方式，要求学生程序输出与期望输出一致。

为了降低格式的影响，系统自动去掉了行首与行尾的空白字符（制表符、空格、控制字符等不可见字符）、以及空白行，进行对比。

完全匹配的限制：

- 打印图形类的题目，例如打印“杨辉三角”，由于“完全匹配”策略自动去掉了行首和行尾的空白字符，对于格式不符合要求的也会判定正确。
- 二进制数据的匹配，“完全匹配”按照文本解析输出内容，可能会将本该比对的数据，解析成为行首行尾的空白字符忽略掉，可能导致错误的输出，被判定正确。

2. 精确匹配

要求学生程序输出与期望输出完全一致，相比“完全匹配”，该策略不会忽略任何空白字符，包括换行。

该匹配策略要求极为严格，与输出的任何不一致都会判定为错误。

适用场景和限制：

- 对于二进制数据的对比，推荐使用“精确匹配”。
- 本地文件上传的测试数据，特别注意换行符的问题，windows平台默认换行符是 `\r\n`，注意将期望输出文件内的换行符转换为 `\n`。。

常见平台报错（不完全一致，意近）：

✧ Accepted（答案正确）

✧ Wrong Answer（答案错误）：

出现该种错误后，一般有两种解决方向：

1. 如果对设计的算法正确性有较大的把握，那么你可以重点考虑是否存在某些特殊数据使程序出现错误，比如边界数据，比如程序中变量出现溢出。
2. 另一种方向，即怀疑算法本身的正确性，那么你就需要重新考虑你的算法设计了。

✧ Time Limit Exceeded（超出时间限制）：

若这种结果出现在你的评判结果里，依然有两种方向可供参考：

1. 假如你确定算法时间复杂度能够符合题目的要求，那么依旧可以检查是否程序可能在某种情况下出现死循环，是否有边界数据可能会让你的代码不按照预想的工作，从而使程序不能正常的结束。
2. 你设计的算法时间复杂度是否已经高于题目对复杂度的要求，如果是这样，那么你需要重新设计更加高效的算法或者对你现行的算法进行一定的优化。

✧ Runtime Error（运行时错误）：

你可以考虑以下几个要点来排除该错误：

1. 程序是否访问了不该访问的内存地址，比如访问数组下标越界。
2. 程序是否出现了除以整数 0，从而使程序异常。
3. 程序是否调用了评判系统禁止调用的函数。
4. 程序是否会出现因为递归过深或其他原因造成的栈溢出。

✧ Compile Error（编译错误）

✧ Memory Limit Exceeded（使用内存超出限制）

造成这种错误的原因主要有两个方面：

1. 你的程序申请过多的内存来完成所要求的工作，即算法空间复杂度过高。
2. 因为程序本身的某种错误使得程序不断的申请内存，例如因为某种原因出现了死循环，使得队列中不断的被放入元素。当然也千万别忽略自己的低级错误，比如在声明数组大小时多打了一个 0。

万 事 如 意

2025/11/14

Let It Go